How to make long traps in bash?

Making Long Traps in Bash

"Long traps" in bash usually means either (1) traps with **multi-line / long command bodies**, or (2) using `trap` to clean up long-running processes. Here are the practical patterns.

1. Multi-line trap with a function (cleanest)

```bash
cleanup() {
    echo "Cleaning up..."
    rm -f "$tmpfile"
    kill "$pid" 2>/dev/null
}
trap cleanup EXIT INT TERM
```

This avoids quoting hell and keeps the trap readable.

2. Inline multi-line trap with a quoted string

```bash
trap '
    echo "Interrupted"
    rm -f /tmp/foo
    exit 130
' INT TERM
```

Single quotes let the body span multiple lines. **Beware**: variables inside single quotes expand at trap-fire time, not definition time — sometimes what you want, sometimes not.

3. Heredoc-style for very long traps

```bash
```

```
trap "$(cat <<'EOF'
    echo "signal caught"
    for f in /tmp/job.*; do rm -f "$f"; done
    exit 1
EOF
)" INT TERM
```

Useful when the body contains many quotes or is generated.

4. Trap on EXIT for long-running jobs

```
bash

#!/usr/bin/env bash
set -euo pipefail

tmpdir=$(mktemp -d)
trap 'rm -rf "$tmpdir"' EXIT

long_running_command --out "$tmpdir/result"
```

`EXIT` fires on normal exit, `exit N`, and fatal errors — ideal for cleanup.

5. Trapping many signals

```
bash

trap 'echo "signal $?"' HUP INT QUIT TERM
```

Note: you **cannot trap** `KILL` or `STOP`.

6. Propagating a long-running child's exit

```
bash

sleep 1000 &
child=$!
trap 'kill "$child" 2>/dev/null; wait "$child" 2>/dev/null; exit 143' TERM INT
wait "$child"
```

Gotchas

Gotchas

- **Signals inside functions**: `trap` set inside a function is global unless you restore it.
- `trap - SIG` resets, `trap '' SIG` ignores.
- **Subshells don't inherit** traps for `INT` / `TERM` in some cases; `EXIT` is inherited.
- Use `trap -p` to inspect current traps.
- Prefer **functions + `EXIT`** over complex inline strings — easier to debug and shellcheck-friendly.

If by "long traps" you meant something else (e.g., `trap` inside long pipelines, or a specific error), tell me more and I'll tailor it.

Should this work?
```
shopt -q globstar
GLOBSTAR_STATE=$?
shopt -q nullglob
NULLGLOB_STATE=$?
cleanup() {
  unset -f usage
  if [ $NULLGLO
```

Short answer: **mostly, but with two real problems and one subtlety.**

What works

- Capturing `$?` right after `shopt -q` is correct — `shopt -q` returns 0 if the option is set, 1 if not.
- Saving state in globals and restoring in `cleanup` is a sound pattern.
- `unset -f cleanup` inside `cleanup` is fine (the function keeps running after unlinking itself).

Problem 1: `RETURN` trap is not what you think

`trap ... RETURN` does **not** fire when the current script/function ends. It fires:

- when a **function returns**, or
- when a **script executed with `source` / `.` returns**.

In a normal top-level script, a `RETURN` trap **never fires**. You almost certainly want `EXIT`:

```
bash

trap 'cleanup' EXIT INT
```

Also, `RETURN` only fires for the function it's set in (or, with `set -T` / `functrace`, for called functions). At top level it's a no-op.

Problem 2: trap fires on INT *and* continues

On `INT`, bash runs the trap and then **resumes** the script (unless the trap itself exits). So `cleanup` runs, unsets `NULLGLOB_STATE` / `GLOBSTAR_STATE` and removes itself — then the script keeps going, and a later signal (or `EXIT`) will run `cleanup` again with those variables gone, so `[ $NULLGLOB_STATE -eq 0 ]` becomes `[ -eq 0 ]` → error.

Two fixes:

Option A — exit from the trap on INT:

```
bash

trap 'cleanup; exit 130' INT
trap 'cleanup' EXIT
```

Option B — make cleanup idempotent / guard the vars:

```
bash

cleanup() {
    [ -n "${NULLGLOB_STATE+x}" ] || return 0
    ...
}
```

Subtlety: `shopt -q` under `set -e`

If the caller has `set -e`, the `shopt -q nullglob` line returning 1 can abort the script before you read `$?`. Safer:

```
bash

shopt -q globstar && GLOBSTAR_STATE=0 || GLOBSTAR_STATE=$?
```

or wrap in `if shopt -q globstar; then ...`. Since you're capturing `$?` immediately, the simple `state=$?` form is fine **only if** `set -e` **is off** at that point.

Cleaned-up version

```bash
shopt -q globstar; GLOBSTAR_STATE=$?
shopt -q nullglob; NULLGLOB_STATE=$?

cleanup() {
    [ -n "${NULLGLOB_STATE+x}" ] || return 0
    unset -f usage_msg allow_decorators
    if [ "$NULLGLOB_STATE" -eq 0 ]; then shopt -s nullglob; else shopt -u nullglob; fi
    if [ "$GLOBSTAR_STATE" -eq 0 ]; then shopt -s globstar; else shopt -u globstar; fi
    unset NULLGLOB_STATE GLOBSTAR_STATE
    unset -f cleanup
}
trap 'cleanup; exit 130' INT
trap 'cleanup' EXIT
```

One more note

If this block lives **inside a function** rather than at script top level, `EXIT` fires when the *shell* exits (not when the function returns), and `RETURN` would fire when that function returns — so the correct choice depends on scope. If it's inside a function and you want restore-on-return, use:

```bash
trap 'cleanup' RETURN
```

...but then also be aware that with `set -T` it fires for *every* function return, which is usually wrong.

Bottom line: switch `RETURN` → `EXIT` (or add `RETURN` only if this is inside a sourced function you actually want to restore on), and make `cleanup` either exit-on-INT or idempotent.